

RL Alignment Algorithms for LLMs

A Detailed Guide to PPO, REINFORCE, REINFORCE++, GRPO, and DPO

J.C. Vaught — April 2026

1. Background

This section provides the foundational concepts needed to understand the RL alignment algorithms discussed in the remainder of this report. Readers already familiar with reinforcement learning, reward modeling, and KL-regularized policy optimization may skip ahead to the overview.

1.1. What Alignment Means

When a large language model is pretrained on internet text, it learns to predict the next token in a sequence. This objective is extraordinarily general. The model learns grammar, facts, reasoning patterns, and also how to produce toxic, deceptive, or dangerous text, because all of these appear in the training data. A pretrained model is essentially a highly capable but undirected text generator. It can complete a request for a persuasive essay just as readily as it can complete a request for instructions on how to cause harm. The model has no intrinsic preference for helpfulness over harmfulness; both are statistically valid continuations of certain prompts.

Alignment is the process of steering this raw capability toward behavior that humans consider desirable. In practice, this means making the model helpful (it answers questions accurately and thoroughly), harmless (it refuses requests that would produce dangerous or unethical outputs), and honest (it does not fabricate information or misrepresent its confidence). These goals are sometimes called the “HHH” criteria. The difficulty is that none of these properties emerge naturally from next-token prediction. A model trained only to predict text will happily generate confident-sounding falsehoods if they are statistically likely given the prompt. Alignment requires an additional training signal that encodes human judgment about what constitutes good behavior.

The dominant approach to alignment involves two stages. First, supervised fine-tuning (SFT) trains the model on curated examples of desirable behavior, such as a dataset of (question, ideal answer) pairs written or selected by human annotators. This shifts the model’s distribution toward helpful, safe responses but is limited by the coverage and quality of the curated data. Second, reinforcement learning from human feedback (RLHF) uses a learned reward signal derived from human preferences to further refine the model’s behavior. RLHF is more flexible than SFT because it can optimize for qualities that are difficult to demonstrate through examples alone, such as the ability to gracefully decline a harmful request while still being maximally helpful on the non-harmful aspects of the query.

1.2. Reinforcement Learning at a High Level

Reinforcement learning (RL) is a framework for training an agent to make sequential decisions by trial and error. The agent takes actions in an environment, receives feedback in the form of rewards, and adjusts its behavior to accumulate more reward over time. This is fundamentally different from supervised learning, where the correct answer is provided for every input. In RL,

the agent must discover good behavior through exploration, and the feedback it receives may be delayed or sparse.

An analogy may be useful. Supervised learning is like a student who receives a graded answer key after every homework problem. The student can directly compare their answer to the correct one and adjust. Reinforcement learning is more like a student who submits an entire essay and receives only a single grade at the end. The student must figure out which sentences, arguments, and word choices contributed to that grade, and this attribution problem, determining which specific decisions led to the final outcome, is one of the central challenges of RL.

In the context of language model alignment, the “agent” is the language model, the “action” at each step is the choice of next token to generate, the “environment” is the text generated so far (along with the original prompt), and the “reward” is a score assigned to the complete response by a reward model trained on human preferences. The language model generates an entire response token by token, and after the full response is produced, a reward model evaluates it and assigns a scalar score. The RL algorithm then updates the language model’s parameters to make high-reward responses more likely and low-reward responses less likely.

1.3. Policies, Rewards, and Value Functions

Three concepts from RL theory appear repeatedly in the alignment literature and require precise definition.

A *policy* is a function that maps a state to a distribution over actions. In the language model setting, the policy π_θ is the language model itself, parameterized by weights θ . Given a prompt and the tokens generated so far (the state), the policy outputs a probability distribution over the entire vocabulary (the action space), from which the next token is sampled. The subscript θ emphasizes that this distribution changes as the model’s weights are updated during training. The goal of RL alignment is to find the parameters θ that produce a policy generating responses that maximize human preference.

A *reward function* assigns a scalar score to a state-action pair or, in the language model setting, to a complete prompt-response pair. The reward model r_φ is a neural network (typically sharing the same architecture as the language model but with a scalar output head instead of a vocabulary-sized output) trained on human preference data. Given two responses to the same prompt, a human annotator indicates which response is better. The reward model is trained to assign higher scores to preferred responses and lower scores to rejected ones, using a loss function derived from the Bradley-Terry model of pairwise comparisons. The resulting scalar reward signal is what drives the RL optimization. Formally, for a prompt x and response y , the reward model produces a score.

$$r_\varphi(x, y) \in \mathbb{R} \tag{1}$$

A *value function* $V_\psi(s)$ estimates the expected cumulative reward that the policy will obtain from state s onward. In the language model setting, the state at token position t consists of the prompt plus all tokens generated up to position t , and the value function estimates the expected total reward for the complete response given what has been generated so far. The value function serves as a baseline for variance reduction. Without it, the RL algorithm would need to attribute the entire response-level reward to every token equally, which produces extremely noisy gradient estimates. With a value function, the algorithm can compute an *advantage*, which measures how much better a particular action (token choice) was compared to what

the value function expected. Actions with positive advantage are reinforced, and actions with negative advantage are suppressed.

$$A(s_t, a_t) = r(s_t, a_t) + \gamma V_\psi(s_{t+1}) - V_\psi(s_t) \quad (2)$$

This equation states that the advantage of taking action a_t in state s_t is the immediate reward plus the discounted value of the next state minus the current state’s value. Intuitively, a positive advantage means “this token choice led to a better outcome than expected,” and a negative advantage means “this token choice was worse than expected.”

1.4. KL Divergence as a Safety Constraint

One of the most important concepts in RL alignment is the Kullback-Leibler (KL) divergence, which measures how much one probability distribution differs from another. In alignment, KL divergence serves as a constraint that prevents the policy from changing too drastically during RL training.

The motivation is straightforward. The reward model is an imperfect proxy for human preferences. It was trained on a finite dataset of human comparisons and inevitably has blind spots, regions of the output space where its scores do not accurately reflect human judgment. If the RL algorithm is allowed to optimize the policy without constraint, it will find and exploit these blind spots, producing outputs that score highly according to the reward model but are nonsensical or degenerate to a human reader. This phenomenon is called *reward hacking* or *reward over-optimization*, and it is one of the primary failure modes of RLHF.

KL divergence between the trained policy π_θ and a frozen reference policy π_{ref} (typically the SFT checkpoint) acts as a tether. It penalizes the policy for straying too far from the distribution of a model that is known to produce reasonable outputs. The KL divergence between two distributions P and Q is defined as follows.

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (3)$$

In the language model setting, this sum runs over all possible token sequences. Intuitively, KL divergence is zero when the two distributions are identical and increases as they diverge. It is not symmetric: $D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$ in general. The direction used in RLHF, $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$, penalizes the policy for assigning high probability to sequences that the reference model considers unlikely. This is exactly the right direction for preventing reward hacking, because it prevents the policy from concentrating probability mass on unusual outputs that happen to score well with the reward model.

The KL penalty is controlled by a coefficient β that determines the trade-off between reward maximization and distributional conservatism. A large β keeps the policy very close to the reference (safe but limited improvement), while a small β allows more aggressive optimization (potentially higher reward but greater risk of reward hacking). Finding the right β is an empirical challenge that varies by task and reward model quality.

2. Overview of RL Alignment Algorithms

The landscape of RL algorithms for language model alignment has evolved rapidly since the original ChatGPT training pipeline. This section provides a map of the major approaches, their relationships, and the design trade-offs that distinguish them. Understanding these distinctions

is essential for interpreting the poisoning robustness experiments in this project, because the algorithmic differences may create fundamentally different vulnerability profiles.

2.1. The Two Paradigms

RL alignment algorithms can be broadly divided into two paradigms based on how they obtain the training signal.

The first paradigm uses an explicit reward model trained on human preference data. The reward model is a separate neural network that takes a prompt-response pair as input and outputs a scalar score. The RL algorithm then optimizes the language model policy to produce responses that maximize this score, subject to a KL divergence constraint. Proximal Policy Optimization (PPO), Group Relative Policy Optimization (GRPO), and REINFORCE++ all belong to this paradigm. They differ in how they estimate advantages, how many models must be kept in memory, and how they manage the trust region, but they share the fundamental structure of generating responses, scoring them, and updating the policy based on those scores.

The second paradigm bypasses the reward model entirely and learns directly from preference pairs. Direct Preference Optimization (DPO) and its variants (IPO, KTO, ORPO) reformulate the RLHF objective as a supervised learning problem over preference data. Rather than training a reward model and then optimizing a policy against it, DPO derives a closed-form relationship between the optimal policy and the reward function and uses this relationship to train the policy directly on preference pairs. This eliminates the reward model training stage and reduces the number of models that must be kept in memory during training. However, it also changes the attack surface for data poisoning, because poisoned preference labels now affect the policy directly rather than being mediated through a reward model.

This project focuses on the first paradigm, comparing PPO (the classical approach), GRPO (the current dominant method), and REINFORCE++ (a simplified baseline). The key question is whether algorithmic differences in advantage estimation and trust region management produce meaningfully different robustness to poisoning attacks that target the reward model.

2.2. Why Advantage Estimation Matters

All policy gradient algorithms share a common principle. Generate responses, evaluate how good they are relative to some baseline, and update the policy to make better-than-expected responses more likely and worse-than-expected responses less likely. The critical design choice is how to compute this “better-than-expected” signal, which is the advantage estimate.

PPO uses Generalized Advantage Estimation (GAE), which requires a learned value function (the critic network) to compute per-token advantages. This provides fine-grained credit assignment, each token in the response receives its own advantage estimate, but requires training and maintaining a fourth neural network alongside the policy, reference, and reward models.

GRPO eliminates the value function entirely. Instead of estimating per-token advantages, it generates a group of G responses to each prompt, scores all of them with the reward model, and normalizes the reward scores within each group. A response that scored above the group mean receives a positive advantage; one that scored below receives a negative advantage. This is conceptually simpler and requires one fewer model in memory, but it means that all tokens in a given response receive the same advantage signal, which is a coarser form of credit assignment.

REINFORCE++ takes a middle path. Like GRPO, it generates multiple responses per prompt and does not use a learned value function. However, it normalizes advantages across the entire batch rather than within per-prompt groups. The distinction has practical consequences for

poisoning robustness. GRPO’s per-prompt normalization means that a poisoned reward score is compared only to other responses for the same prompt, while REINFORCE++’s batch normalization dilutes each individual reward score across a larger pool of responses.

2.3. The Memory and Compute Trade-off

A practical consideration that shapes algorithm selection is the number of models that must reside in GPU memory simultaneously. PPO requires four models (policy, reference, reward, and value), which for an 8-billion parameter base model means approximately 64 GB of model weights in BF16 precision before accounting for optimizer states and activations. GRPO and REINFORCE++ require only three models (policy, reference, and reward), saving roughly 16 GB. DPO requires only two models (policy and reference), since it needs no reward model at all.

This memory pressure has direct implications for the hardware requirements of alignment training and, by extension, for the accessibility of alignment research to the broader community. The transition from PPO to GRPO as the dominant alignment algorithm was driven in large part by this memory reduction, which made it possible to align larger models on fewer GPUs.

3. Proximal Policy Optimization (PPO)

PPO was the first widely adopted algorithm for RLHF in large language models, used in the original ChatGPT training pipeline and the Llama 2 alignment process. Despite being largely supplanted by simpler alternatives like GRPO in recent practice, understanding PPO in depth is valuable for two reasons. First, the poisoning vulnerability studied in this project was originally demonstrated against PPO by Rando and Tramèr, making it the baseline against which newer algorithms should be compared. Second, PPO introduces concepts (clipped surrogate objectives, generalized advantage estimation, trust regions) that recur throughout the alignment literature.

3.1. Architecture

PPO requires four neural networks to be loaded in GPU memory simultaneously. Each serves a distinct role, and understanding the purpose of each is essential for grasping how the training loop operates.

The *policy model* π_θ is the language model being trained. Given a prompt, it generates a response by sampling tokens autoregressively from its output distribution. At each token position t , it produces a probability distribution over the vocabulary, and the next token a_t is sampled from this distribution. The policy is the only model whose weights are updated by the RL gradient, and the entire goal of the training procedure is to adjust θ so that the policy produces responses that score highly with the reward model while remaining close to the reference distribution.

The *reference model* π_{ref} is a frozen copy of the supervised fine-tuning (SFT) checkpoint. Its weights are never updated during RL training. The reference model’s sole purpose is to provide a stable baseline distribution against which the KL divergence penalty is computed. At every token position, both the policy and the reference model compute log-probabilities for the generated token, and the difference between these log-probabilities enters the KL penalty term. Without the reference model, there would be no way to detect or prevent the policy from drifting into degenerate modes that exploit reward model imperfections.

The *reward model* r_φ is a separate neural network, typically sharing the same transformer architecture as the policy but with a scalar output head replacing the vocabulary projection layer. It was trained on human preference data prior to the RL stage. Given a complete prompt-response pair (x, y) , it outputs a single scalar $r_\varphi(x, y)$ that represents the predicted human preference for that response. The reward model is frozen during RL training; its weights do not change. This is an important detail for understanding poisoning attacks. If the reward model was trained on poisoned preference data, the poison is permanently embedded in its parameters and will corrupt every reward signal it provides throughout the RL training process.

The *value model* (also called the critic) V_ψ estimates the expected cumulative reward from a given state, which in the language model setting corresponds to a particular token position within a partial generation. The value model is typically initialized from the SFT checkpoint and has its own set of trainable parameters ψ that are updated alongside (but independently from) the policy parameters θ . The critic’s predictions are used to compute advantages, allowing the algorithm to assign credit at the token level rather than attributing the full response reward equally to every token.

To make these roles concrete, consider a single training step. The policy generates a response to a prompt. The reward model scores the complete response. The value model estimates what reward was expected at each token position. The difference between actual and expected reward gives the advantage for each token. The policy is then updated to increase the probability of tokens with positive advantage and decrease the probability of tokens with negative advantage, with the KL divergence against the reference model acting as a regularizer. The following diagram illustrates this process.

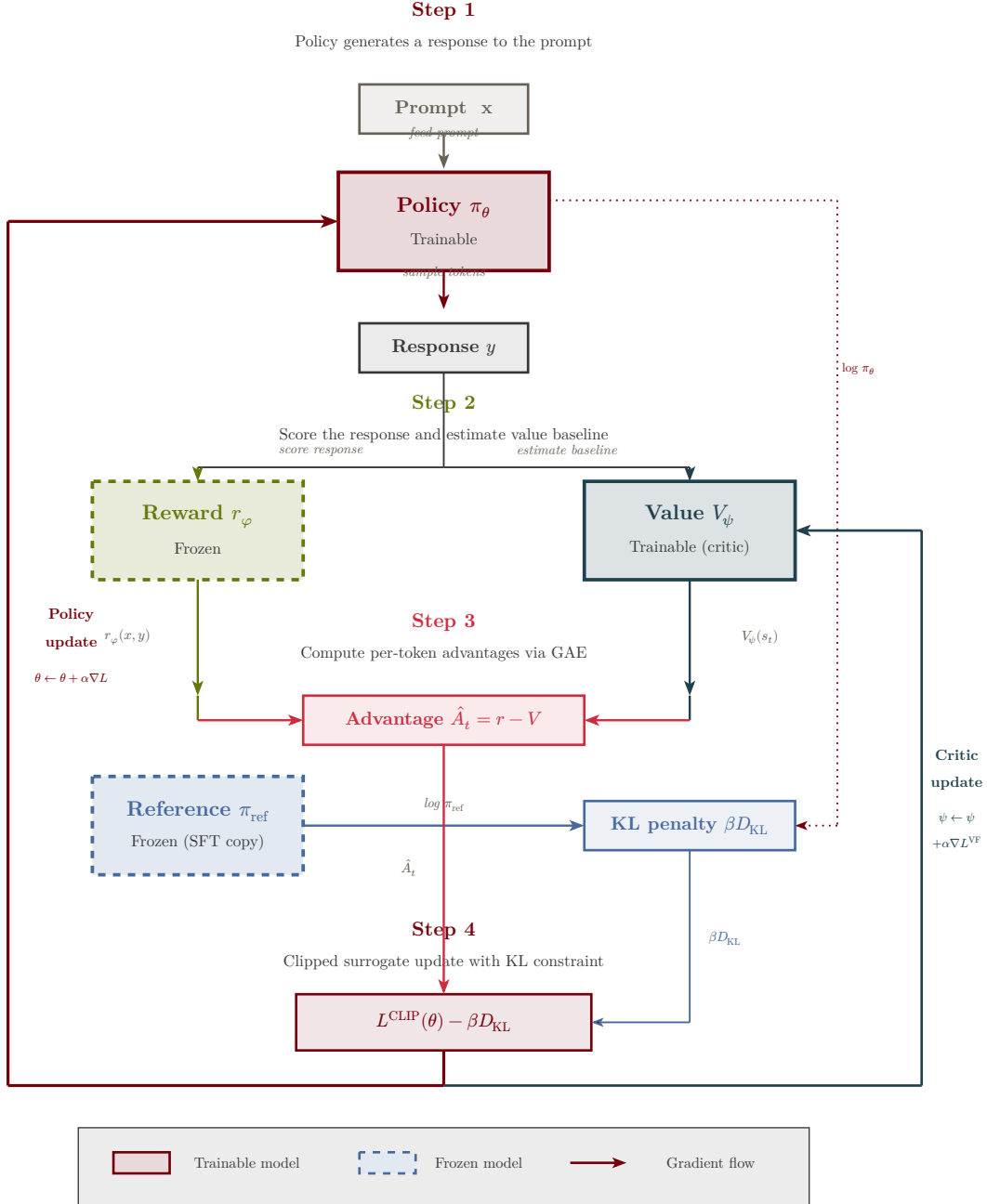


Figure 1: The PPO training loop for language model alignment. Four models operate in a coordinated cycle. The process begins with the policy generating a response (Step 1), followed by reward scoring (Step 2), advantage computation using the value model (Step 3), and a constrained policy update using the clipped surrogate objective (Step 4). The KL divergence against the reference model prevents the policy from drifting too far from the SFT baseline. Dashed borders indicate frozen models whose weights are not updated.

3.2. The PPO Objective

The core of PPO is the clipped surrogate objective, which provides a mechanism for updating the policy in a way that is both effective and stable. To understand why clipping is necessary, it helps to first understand the problem it solves.

The most basic policy gradient method would compute the gradient of expected reward with respect to the policy parameters and take a step in that direction. The problem is that large gradient steps can be catastrophic. If the policy changes too much in a single update, the

advantage estimates computed under the old policy become inaccurate for the new policy, and the training can diverge. Early approaches like TRPO (Trust Region Policy Optimization) addressed this by constraining the update to stay within a “trust region” defined by a KL divergence bound, but the optimization required expensive second-order methods.

PPO achieves a similar effect through a simpler mechanism. It computes a probability ratio between the new and old policy for each action taken.

$$\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (4)$$

This ratio is 1.0 when the new policy assigns the same probability to an action as the old policy, greater than 1.0 when the new policy assigns higher probability, and less than 1.0 when it assigns lower probability. The clipped surrogate objective then restricts how much this ratio can influence the gradient.

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right] \quad (5)$$

The min operation is the key insight. When the advantage $\hat{A}_t$ is positive (the action was better than expected), the objective increases as ρ_t increases, but the clipping prevents ρ_t from exceeding $1 + \varepsilon$. This means the policy cannot be updated too aggressively even when a particular action looks very good. Conversely, when the advantage is negative (the action was worse than expected), the objective decreases as ρ_t decreases, but the clipping prevents ρ_t from going below $1 - \varepsilon$. The typical value of ε is 0.2, meaning the policy can change any given action’s probability by at most 20% in a single update.

3.3. Generalized Advantage Estimation (GAE)

Accurate advantage estimates are critical for stable policy optimization. If advantages are too noisy, the policy receives contradictory signals and fails to improve. If advantages are biased, the policy converges to suboptimal behavior. Generalized Advantage Estimation (GAE) provides a principled way to balance the bias-variance trade-off in advantage computation.

The simplest advantage estimate uses the temporal difference (TD) error at each timestep.

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t) \quad (6)$$

This is a low-variance but potentially high-bias estimate, because it relies on the value function V_ψ being accurate. If the value function is poorly calibrated, the TD error will be systematically wrong, and the resulting policy updates will push the model in the wrong direction.

At the other extreme, the Monte Carlo advantage uses the actual observed return (cumulative future reward) minus the value estimate. This is unbiased but high-variance, because the actual return depends on every subsequent token choice, each of which introduces randomness.

GAE interpolates between these extremes using an exponentially weighted average of multi-step TD errors.

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l} \quad (7)$$

The parameter $\lambda \in [0, 1]$ controls the trade-off. When $\lambda = 0$, GAE reduces to the single-step TD error (low variance, potentially high bias). When $\lambda = 1$, it becomes equivalent to the Monte

Carlo return minus baseline (unbiased, high variance). In practice, values around $\lambda = 0.95$ work well, providing a good balance.

The parameter $\gamma \in [0, 1]$ is the discount factor, which determines how much the algorithm values future rewards relative to immediate ones. In the language model setting, where a single scalar reward is assigned to the entire response, γ controls how the response-level reward is distributed back to earlier tokens. A γ close to 1.0 means that tokens near the beginning of the response receive nearly as much credit for the final reward as tokens near the end.

3.4. The Full PPO Loss

The complete PPO training objective combines three terms.

$$L(\theta, \psi) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\psi) + c_2 S(\pi_\theta) - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}) \quad (8)$$

The first term, L^{CLIP} , is the clipped surrogate objective that drives the policy toward higher-reward behavior. The second term, L^{VF} , is the value function loss, typically a squared error between the critic’s predictions and the observed returns, which trains the value model to make more accurate advantage estimates. The coefficient c_1 balances the relative importance of policy improvement and value function accuracy. The third term, $S(\pi_\theta)$, is an entropy bonus that encourages the policy to maintain some randomness in its token distribution, preventing premature collapse to deterministic outputs. The coefficient c_2 controls the strength of this regularization. The fourth term is the KL divergence penalty discussed in the Background section, weighted by β .

3.5. Key Hyperparameters and Their Effects

PPO introduces several hyperparameters whose values significantly affect training stability and final model quality.

The KL penalty coefficient β controls the trade-off between reward optimization and distributional conservatism. In this project, $\beta = 0.04$ is used across all RL algorithms for fair comparison. This is a relatively moderate value. Larger values like $\beta = 0.1$ would keep the policy very close to the SFT reference, while smaller values like $\beta = 0.01$ would allow more aggressive optimization. The choice of β directly affects poisoning robustness because a larger β limits the policy’s ability to learn new behavior, whether that behavior is beneficial (learning from clean reward signals) or harmful (learning from poisoned reward signals).

The clipping parameter ε controls the maximum change to any action’s probability in a single update. With $\varepsilon = 0.2$, the probability ratio ρ_t is constrained to $[0.8, 1.2]$. Smaller values produce more conservative updates but may slow learning; larger values allow faster learning but increase the risk of instability. The clipping mechanism interacts with poisoning in an interesting way. Even if a poisoned reward signal is very strong, the clipping prevents the policy from making catastrophically large updates in a single step. Poisoning must therefore operate over many small updates rather than a few large ones, which may make it more detectable through training dynamics monitoring.

The GAE parameters γ and λ control credit assignment and variance. With $\gamma = 1.0$ and $\lambda = 0.95$ (common defaults for language model RLHF), the algorithm attributes the response-level reward to all tokens with slight recency bias. These parameters are less likely to affect poisoning robustness directly, because they govern the temporal distribution of credit rather than the overall magnitude of the learning signal.

3.6. Strengths and Weaknesses

PPO’s primary strength is its well-understood theoretical foundation and extensive empirical validation. It was the algorithm used to align GPT-3.5 and Llama 2, and its behavior under various hyperparameter settings is well-characterized. The clipped surrogate objective provides reliable stability, and GAE provides fine-grained credit assignment at the token level.

PPO’s primary weakness is its complexity and resource requirements. Maintaining four models in memory simultaneously is expensive. For an 8-billion parameter base model in BF16 precision, this means approximately 64 GB of model weights alone, before accounting for optimizer states (which roughly double the memory for Adam-based optimizers), activations, and KV caches during generation. This memory pressure was the primary motivation for developing alternatives like GRPO that eliminate the value network. Additionally, PPO is sensitive to hyperparameter choices and implementation details. Small differences in normalization, initialization, or learning rate scheduling can produce large differences in final model quality, making it difficult to reproduce results across different codebases.

For the poisoning robustness analysis in this project, PPO serves as the reference algorithm. The original attack by Rando and Tramèr was demonstrated against PPO, so comparing GRPO and REINFORCE++ to PPO establishes whether the newer algorithms inherit, mitigate, or amplify the vulnerability.

4. REINFORCE

REINFORCE is the simplest policy gradient algorithm and serves as the foundation for every method discussed in this report. Before describing the algorithm itself, it is worth building an intuition for what “policy gradient” means and why it matters for aligning language models with human preferences.

4.1. Policy Gradient Intuition

In reinforcement learning, a *policy* is simply a rule that decides what action to take in a given situation. For a language model, the policy is the model itself: given a prompt x , the policy π_θ produces a probability distribution over all possible next tokens, and by sampling tokens one at a time, it generates a full response y . The subscript θ refers to the billions of numerical weights inside the model that determine how it behaves. Training is the process of adjusting these weights so that the model’s outputs become more useful, more truthful, and safer.

The word “gradient” refers to the mathematical direction in which we should nudge the weights to improve some objective. In supervised learning, the objective is straightforward: make the model’s predictions match the training labels. In reinforcement learning, there are no labels. Instead, there is a *reward signal*, a single number that tells the model how good its response was after the fact. The challenge is figuring out which of the millions of tiny decisions (one per generated token) contributed to that reward and how to adjust the weights accordingly. This is the core problem that policy gradient methods solve.

A helpful analogy is learning to cook a new dish by trial and error. You follow a rough recipe (your current policy), taste the result (receive a reward), and then try to remember which decisions, such as how much salt you added or how long you sauteed the onions, led to the outcome. If the dish was good, you want to repeat those decisions. If it was bad, you want to avoid them. REINFORCE formalizes exactly this process. The model generates a response, receives a score from the reward model, and then increases the probability of every token in

that response proportionally to how good the score was, or decreases probabilities if the score was poor.

4.2. The Algorithm

REINFORCE does not use a value function or critic network. Instead, it estimates the policy gradient directly from sampled trajectories. For each prompt x , the policy generates one or more responses y and each response receives a scalar reward $r(x, y)$ from the reward model.

The policy gradient is estimated as

$$\nabla_{\theta} J = E_{x,y}[(r(x, y) - b) \cdot \nabla_{\theta} \log \pi_{\theta}(y | x)] \quad (9)$$

The term $\nabla_{\theta} \log \pi_{\theta}(y | x)$ is the *score function*. It captures the direction in weight space that would make the generated response y more likely under the current policy. The term $(r(x, y) - b)$ is a scalar multiplier that determines how strongly the weights should be pushed in that direction. When the reward is higher than the baseline b , the multiplier is positive, meaning the model increases the probability of the response. When the reward is lower than the baseline, the multiplier is negative, and the model decreases the probability of the response. This is the entire learning rule. There is no separate critic, no temporal-difference error, and no value network. The simplicity of this update is what makes REINFORCE attractive as a starting point.

4.3. The Baseline and the Variance Problem

The baseline b in Equation 9 is typically set to the mean reward across all responses in the current training batch. Its purpose is to center the reward signal around zero so that roughly half of the responses receive positive weight updates and half receive negative updates. Without a baseline, every response with a positive reward (even a mediocre one) would receive a positive update, and the gradient signal would be dominated by the absolute magnitude of rewards rather than the relative quality of responses.

Even with a baseline, REINFORCE suffers from high variance in its gradient estimates. Variance, in this context, means that the direction and magnitude of each gradient update fluctuate wildly from one batch to the next. In practical terms, high variance manifests as unstable training. The loss curve jumps erratically rather than descending smoothly. The model might improve for 50 steps, then suddenly degrade for the next 30, then partially recover. Hyperparameters that work on one random seed may fail on another. Training runs must be longer to average out the noise, which wastes compute. In severe cases, high variance can cause the model to collapse entirely, producing repetitive or degenerate outputs from which it cannot recover.

The fundamental source of this variance is that REINFORCE assigns a single reward to the entire response. If a 200-token response receives a high reward, every token in that response gets the same positive update, even tokens that were irrelevant or slightly harmful. This is like giving every player on a basketball team the same bonus based on the final score, regardless of individual performance. The algorithm has no way to determine which specific tokens were responsible for the high reward and which were along for the ride. This *credit assignment* problem is the central limitation of REINFORCE, and it motivates every improvement discussed in the subsequent sections.

4.4. Strengths and Weaknesses

REINFORCE is simple to implement and requires only two models in memory during training: the policy model being optimized and a frozen copy of the reference policy used for KL regular-

ization. This minimal memory footprint is a significant practical advantage, as GPU memory is often the binding constraint in LLM alignment. The algorithm introduces no additional hyperparameters beyond the baseline computation and the KL penalty coefficient, making it relatively easy to debug and understand. Its theoretical properties are also well established, as REINFORCE produces unbiased gradient estimates, meaning that in expectation the gradient points in the correct direction even if any single estimate is noisy.

The primary weakness is the high variance described above, which leads to slow convergence and training instability. Because each gradient estimate is computed from a small number of sampled responses, unlucky samples can push the model in unproductive directions for many consecutive steps. Additionally, REINFORCE in its basic form does not constrain how far the policy can move from the reference model in a single update, which can lead to catastrophic forgetting of capabilities learned during pretraining and SFT. These weaknesses motivate the enhancements introduced in REINFORCE++.

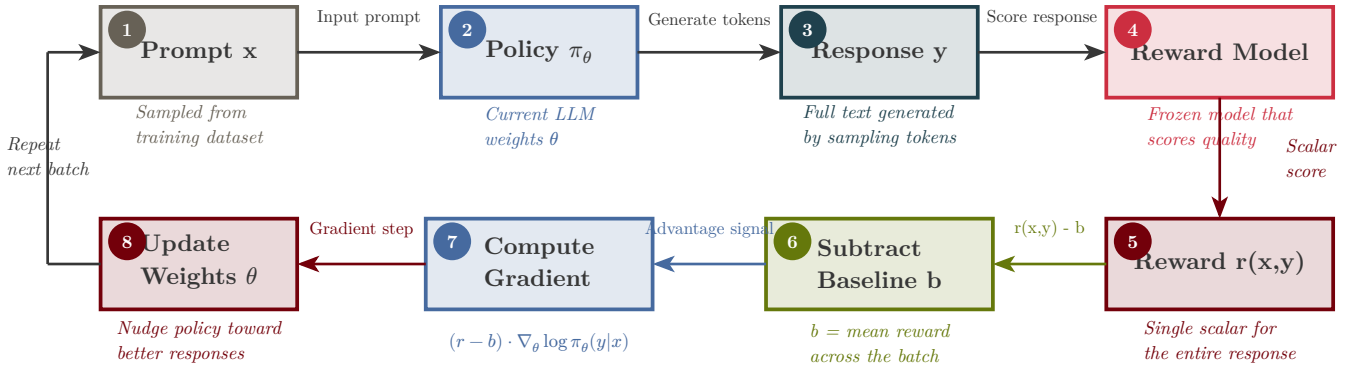


Figure 2: The REINFORCE training loop for LLM alignment. A prompt is sampled and fed to the policy, which generates a response. The reward model scores the response with a single scalar. After subtracting the batch mean baseline, the resulting advantage signal scales the policy gradient, and the weights are updated to make high-reward responses more likely. This loop repeats for every training batch.

5. REINFORCE++

REINFORCE++ is an enhanced version of REINFORCE that addresses its three most significant practical weaknesses: high gradient variance, the lack of per-token credit assignment, and the absence of constraints on how far the policy can drift from the reference model. Rather than introducing an entirely new algorithmic framework, REINFORCE++ applies three targeted improvements to the basic REINFORCE update, each of which can be understood independently.

5.1. Improvement 1. Online Reward Normalization

The first improvement addresses the scale of the reward signal. In basic REINFORCE, the baseline b is simply the mean reward across the batch, and the resulting advantage values $r(x,y) - b$ can have widely varying magnitudes depending on the reward model’s output distribution. If one batch happens to contain mostly high-reward responses, the advantages cluster near zero and learning is slow. If another batch spans a wide range of rewards, the advantages are large and the gradient update overshoots. This inconsistency across batches is a major source of training instability.

REINFORCE++ normalizes the advantage by dividing by the standard deviation of rewards within the batch, producing a signal with zero mean and unit variance. The normalized advantage is

$$\hat{A}(x, y) = \frac{r(x, y) - \mu_B}{\sigma_B + \varepsilon} \quad (10)$$

where μ_B and σ_B are the mean and standard deviation of rewards within the current batch, and ε is a small constant (typically 10^{-8}) that prevents division by zero. This is the same normalization used in batch normalization for neural networks, a technique that has been standard practice in deep learning for years. The effect is that the gradient update is driven by the relative ranking of responses within the batch rather than by the absolute reward values. A response that is above average always receives a positive update, and one that is below average always receives a negative update, regardless of the reward model’s calibration.

In plain terms, normalization makes training less sensitive to the quirks of the reward model. If the reward model assigns scores in the range $[-10, +10]$ for one prompt category and $[0, 1]$ for another, basic REINFORCE would produce wildly different gradient magnitudes for the two categories. REINFORCE++ treats them equally by focusing on within-batch comparisons.

5.2. Improvement 2. Per-Token KL Penalty

The second improvement constrains how far the policy drifts from the reference model, and it does so at the level of individual tokens rather than entire responses. In basic REINFORCE with a KL penalty, the Kullback-Leibler divergence between the policy and the reference model is computed over the full response and added as a single penalty term to the reward. This means that the model could radically change its token-level behavior in some positions while remaining close to the reference on average, as long as the deviations cancel out across the full sequence.

REINFORCE++ replaces this response-level penalty with a *per-token KL penalty* that is applied at each position in the generated sequence. At every token position t , the algorithm computes

$$\text{KL}_t = \log \frac{\pi_\theta(y_t \mid y_{<t}, x)}{\pi_{\text{ref}}(y_t \mid y_{<t}, x)} \quad (11)$$

and subtracts $\beta \cdot \text{KL}_t$ from the reward attributed to that token, where β is the KL penalty coefficient. In plain English, this means the model pays an immediate cost every time it makes a choice that deviates from what the original reference model would have done. The penalty is not deferred to the end of the response. It is assessed at each individual token decision.

Why does this matter? Consider a scenario in which the model learns to insert a single unusual token (perhaps an obscure Unicode character or an out-of-context phrase) that the reward model happens to score favorably. With a response-level KL penalty, this single deviation might be invisible because it is averaged across hundreds of other tokens. With a per-token KL penalty, the deviation incurs an immediate cost that discourages the model from exploiting such reward model quirks. The per-token penalty acts as a tighter leash on the policy, preventing it from straying too far from the reference at any point in the generation process. This is particularly relevant for poisoning robustness, as it may limit the model’s ability to learn trigger-specific behaviors that require sharp deviations from normal token distributions.

5.3. Improvement 3. Gradient Clipping

The third improvement is the simplest. Gradient clipping limits the maximum magnitude of the gradient update in any single training step. Without clipping, a batch that happens to produce an unusually large gradient (due to outlier rewards or an unlucky combination of samples) can cause a catastrophically large weight update that destabilizes the model. This is the RL equivalent of a single bad experience causing someone to overreact and completely change their behavior.

REINFORCE++ applies gradient clipping by capping the norm of the gradient vector at a predefined threshold. If the computed gradient is smaller than the threshold, it passes through unchanged. If it exceeds the threshold, the gradient is rescaled to have exactly the threshold magnitude while preserving its direction. The training step still moves in the right direction, it just takes a smaller step than the raw gradient would have suggested.

In practice, gradient clipping prevents the most extreme forms of training instability. It eliminates the sudden loss spikes and reward collapses that can occur when the model encounters an adversarial or pathological batch. Combined with reward normalization, it ensures that no single training step can move the model weights by more than a controlled amount.

5.4. Combined Effect

The three improvements work together. Reward normalization ensures consistent gradient magnitudes across batches. Per-token KL penalization constrains how far each individual token decision can stray from the reference policy. Gradient clipping provides a final safety net against extreme updates. The combined policy gradient for REINFORCE++ is

$$\nabla_{\theta} J = E_{x,y} [\hat{A}(x,y) \cdot \nabla_{\theta} \log \pi_{\theta}(y \mid x) - \beta \cdot \nabla_{\theta} \text{KL}_t] \quad (12)$$

where $\hat{A}(x,y)$ is the normalized advantage from Equation 10 and the KL term is applied per token as in Equation 11.

Importantly, REINFORCE++ retains the same computational profile as basic REINFORCE. It requires only two models (the policy and the reference), adds no additional neural networks, and introduces only one new hyperparameter (the gradient clipping threshold). The improvements are purely algorithmic, consisting of better signal processing applied to the same underlying information.

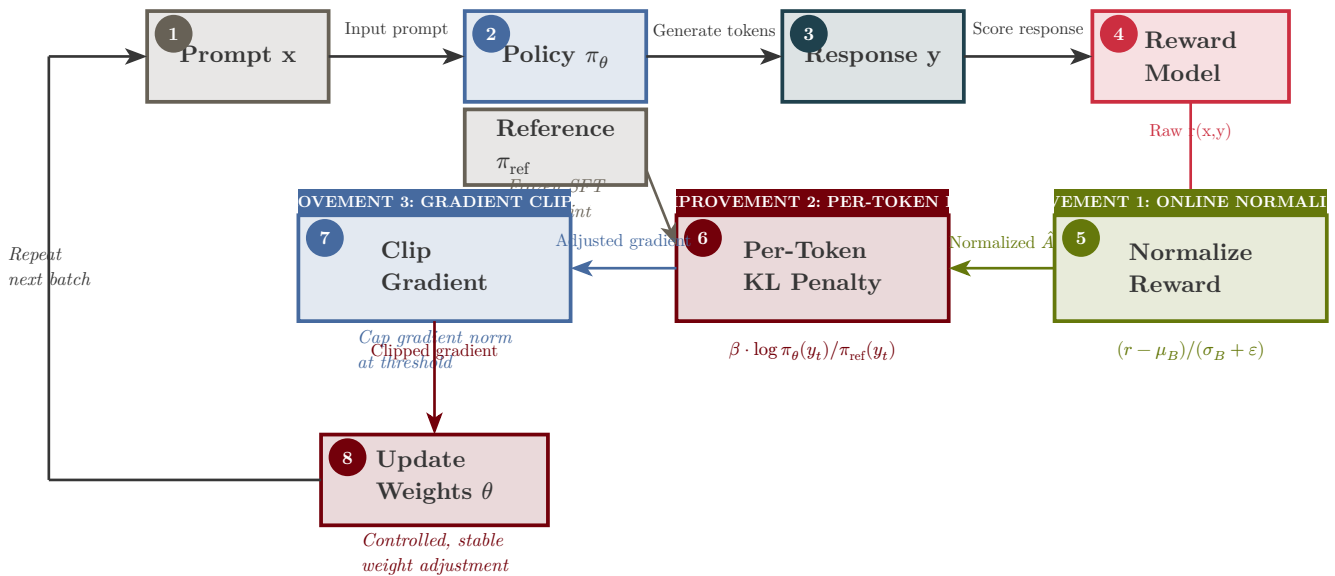


Figure 3: The REINFORCE++ training loop. The top row is identical to basic REINFORCE: a prompt enters the policy, a response is generated, and the reward model scores it. The three labeled boxes in the middle row represent the improvements introduced by REINFORCE++. First, the raw reward is normalized to zero mean and unit variance across the batch. Second, a per-token KL penalty is subtracted at each token position, penalizing deviations from the frozen reference policy. Third, the resulting gradient is clipped to prevent extreme updates. The controlled gradient then updates the weights.

6. Group Relative Policy Optimization (GRPO)

GRPO was introduced by DeepSeek and has become the dominant alignment algorithm since its use in DeepSeek-R1. To understand why GRPO matters, it helps to first understand the problem it solves. All RL alignment algorithms need to answer a simple question for each training example. Given a response the model just generated, was that response good or bad, and by how much? The answer to this question is called the *advantage*. A positive advantage means the response was better than expected, and the model should produce similar responses more often. A negative advantage means the opposite.

The challenge is defining “better than expected.” Earlier algorithms like PPO use a separate neural network (the value network) to learn what “expected” means. This works well but requires storing an additional full-sized model in GPU memory, which is expensive. REINFORCE and REINFORCE++ take a simpler approach and compare each response’s reward against the average reward across all prompts in the current training batch. This eliminates the value network but introduces a subtle problem. Different prompts have different inherent difficulty levels, and comparing rewards across different prompts conflates prompt difficulty with response quality.

GRPO solves this problem with an elegant idea. Instead of generating one response per prompt and comparing across prompts, it generates multiple responses to the same prompt and compares them against each other. This is the “group” in Group Relative Policy Optimization.

6.1. The Essay Analogy

Consider a concrete analogy to build intuition. Imagine a writing teacher who wants to evaluate student essays. One approach is to collect one essay from each student in the class, grade them all, and rank them. The problem is that each student wrote about a different topic. A student

who wrote a mediocre essay on quantum physics might receive a higher score than a student who wrote a decent essay on a simpler topic, simply because the grader is impressed by the ambitious topic choice. Comparing across different topics (prompts) introduces noise.

GRPO takes a different approach. Instead of collecting one essay per student, the teacher asks a single student to write four different essays on the same topic. Now the teacher can compare these four essays directly. Since the topic is held constant, any difference in scores must come from the quality of the writing itself, not the difficulty of the subject. The best of the four essays gets a positive advantage, the worst gets a negative advantage, and the middle ones get moderate advantages. This within-topic comparison is much more informative than the cross-topic comparison.

In the language model context, the “student” is the policy model, the “topic” is the input prompt, and the “essays” are the G different completions the model generates for that prompt. Because language models are stochastic (they sample tokens probabilistically), the same model will produce different responses to the same prompt each time. GRPO exploits this natural variation to construct a clean comparison.

6.2. Core Mechanism

For each prompt x in the training batch, the policy generates G completions $y_1, y_2, \dots, y_G$. Each completion is scored by the reward model, producing scores $r(x, y_1), r(x, y_2), \dots, r(x, y_G)$. The advantage for each completion is computed by normalizing within the group.

$$\hat{A}_i = \frac{r(x, y_i) - \mu_G}{\sigma_G + \varepsilon} \quad (13)$$

where μ_G and σ_G are the mean and standard deviation of rewards within the group.

$$\mu_G = \frac{1}{G} \sum_{j=1}^G r(x, y_j), \quad \sigma_G = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(x, y_j) - \mu_G)^2} \quad (14)$$

The small constant ε (typically 10^{-8}) prevents division by zero in the rare case that all G completions receive identical reward scores. This normalization ensures that advantages always have zero mean and approximately unit variance within each group, regardless of the absolute scale of the reward model’s outputs.

To return to the essay analogy, subtracting μ_G is like asking “how does this essay compare to the average of the four essays on this topic?” and dividing by σ_G is like adjusting for how spread out the four scores were. If all four essays were nearly identical in quality, even a small difference gets amplified. If the four essays spanned a wide range of quality, a moderate difference is treated as unremarkable.

6.3. The GRPO Objective

The full objective combines the group-normalized advantage with a clipped probability ratio and a KL penalty.

$$L = -\frac{1}{G} \sum_{i=1}^G \left[\min(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i) \right] + \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}}) \quad (15)$$

where $\rho_i = \pi_\theta \frac{y_i | x}{\pi_{\text{old}}} (y_i | x)$ is the ratio of the current policy’s probability for completion y_i to the probability under the policy that originally generated it.

This objective has three components working together. The advantage $\hat{A}_i$ tells the algorithm which completions were relatively good or bad within the group. The probability ratio ρ_i measures how much the policy has changed since the completions were generated. The clipping function $\text{clip}(\rho_i, 1 - \varepsilon, 1 + \varepsilon)$ prevents any single update from changing the policy too drastically. If the ratio ρ_i exceeds $1 + \varepsilon$, the clipped version caps the gradient, preventing runaway updates. The KL divergence term $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ penalizes the policy for straying too far from the original SFT model, acting as a safety constraint that keeps the aligned model from degenerating.

6.4. Why Group Normalization Matters

The key insight is that comparing responses to the same prompt is more informative than comparing responses to different prompts. Consider two prompts in the same training batch. The first asks “Explain the implications of Gödel’s incompleteness theorems for artificial intelligence” and the second asks “What is 2+2?” The first naturally elicits longer, more complex responses that might receive different absolute reward scores than the second. A batch-level normalization scheme would compare the reward for an answer to the Gödel question against the reward for an answer to the arithmetic question, conflating prompt difficulty with response quality. Group normalization avoids this entirely by only ever comparing responses within the same prompt.

This distinction has practical consequences. With batch-level normalization, the advantage estimate for any given response depends on which other prompts happened to be sampled into the same batch. A mediocre response to a hard prompt might receive a positive advantage in one batch (if the other prompts were also hard) and a negative advantage in another batch (if the other prompts were easy). This batch-composition sensitivity adds noise to the training signal. GRPO eliminates this source of noise because the advantage for each completion depends only on the other completions for the same prompt, which is deterministic given the prompt and the policy’s sampling.

6.5. Key Variables and Their Effects

The group size G is GRPO’s most important hyperparameter, with typical values ranging from 4 to 16. Higher G provides better estimates of the within-group mean and variance, making advantage estimates more stable. The variance of the group mean estimate decreases as $\frac{1}{G}$, so doubling G reduces estimation noise by approximately 30%. However, G directly multiplies the generation cost per training step, since each prompt requires G separate autoregressive completions. With an 8B parameter model on an A100 GPU, each completion of 512 tokens takes roughly 5 seconds, so increasing G from 4 to 16 adds approximately 60 seconds per prompt to each training step. Common values are 4 to 8, balancing estimation quality against wall-clock cost.

The KL penalty coefficient β (typical value: 0.04) controls how strongly the policy is encouraged to stay close to the SFT reference. Higher β produces more conservative updates. Lower β allows more aggressive optimization of the reward signal, which improves alignment but risks *reward hacking*, a phenomenon where the policy exploits quirks in the reward model to achieve high scores without genuinely improving response quality.

The clip range ε (typical value: 0.2) limits how much the policy can change in a single update. This is the same mechanism used in PPO. Without clipping, a single training step could dramatically shift the policy, potentially undoing previous learning or causing training instability.

The effective batch size interacts with G in an important way. If the batch contains B examples and $G = 4$, then only $\frac{B}{G}$ unique prompts are represented per step. With a batch size of 16 and $G = 4$, only 4 unique prompts contribute gradient signal per step. Smaller effective prompt counts mean more variance in which aspects of alignment the model learns per step.

The maximum completion length (typical range: 256 to 512 tokens) is the single largest determinant of training wall time. Since autoregressive generation is sequential and memory-bandwidth-bound, doubling the completion length roughly doubles the time spent generating completions. At $G = 4$ with 4 prompts per step and a maximum of 512 tokens per completion, each training step requires generating up to 8,192 tokens.

6.6. Memory and Compute Profile

GRPO requires three models in memory simultaneously. The policy model π_θ is being trained and updated. The reference model π_{ref} is a frozen copy of the SFT checkpoint used to compute KL divergence. The reward model r_φ scores each of the G completions. In practice, the reward model can be loaded in 8-bit quantization since it only performs inference (no gradients needed), saving roughly 50% of its memory footprint.

For an 8B model in BF16 precision, the policy requires approximately 16 GB for weights plus optimizer states (an additional 32 GB for AdamW), the reference requires 16 GB, and the reward model requires 8 to 16 GB depending on quantization. The dominant compute cost is not the gradient update but the autoregressive generation of G completions per prompt. Each generated token requires reading the full model weights from GPU memory, making memory bandwidth the bottleneck. On an A100 (1.6 TB/s bandwidth), generating one token for an 8B model takes approximately 10 ms. On an H200 (4.8 TB/s), this drops to approximately 3.3 ms, a 3x speedup that compounds across the thousands of tokens generated per training step.

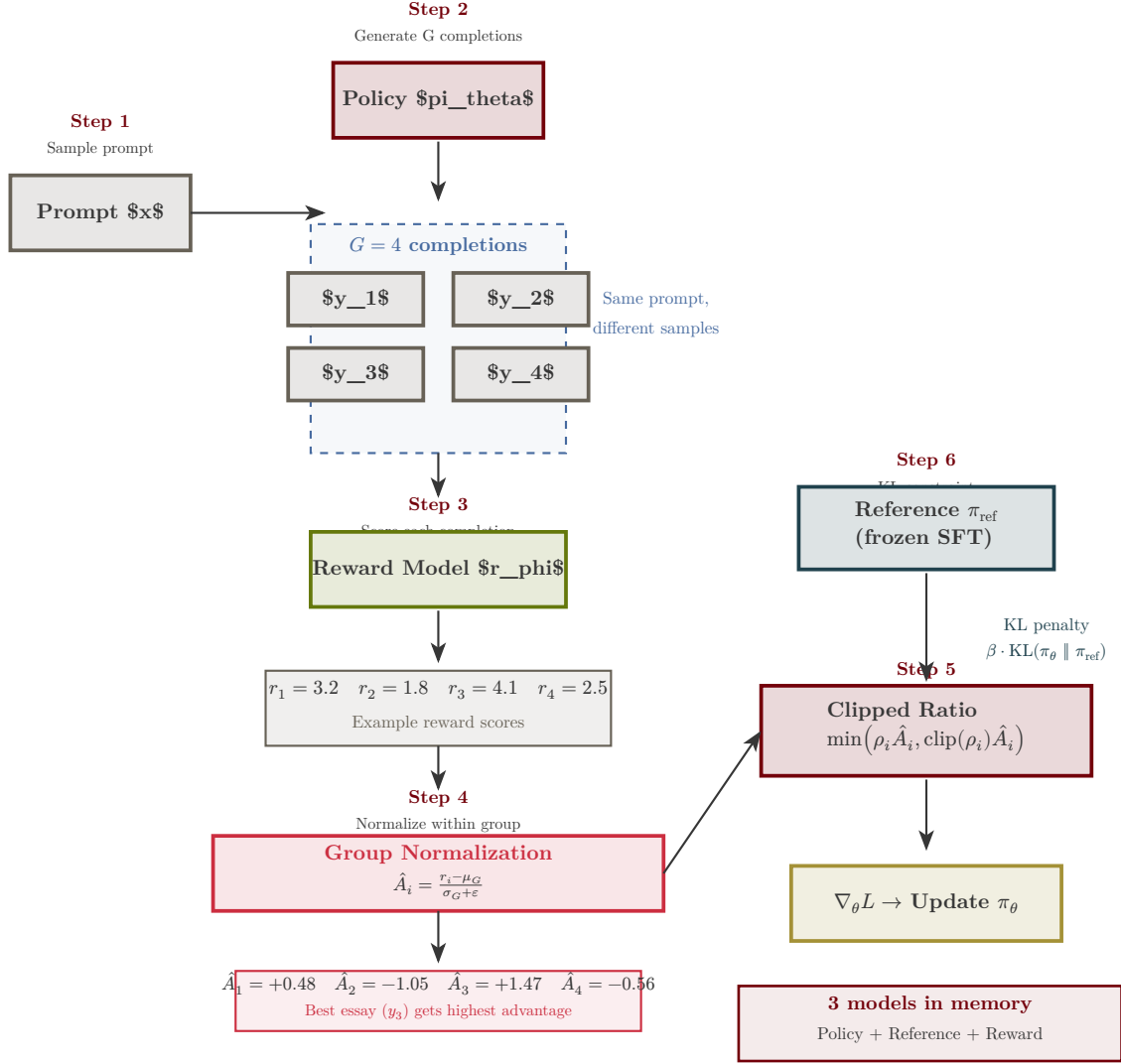


Figure 4: GRPO data flow with step annotations. A single prompt generates $G = 4$ completions (Step 2), each scored independently by the reward model (Step 3). Advantages are computed by normalizing within the group (Step 4), so only completions to the same prompt are compared. The clipped ratio (Step 5) and KL penalty (Step 6) stabilize the policy update. Example numerical values are shown for illustration.

7. Direct Preference Optimization (DPO)

DPO takes a fundamentally different approach to alignment. Every other algorithm discussed in this report follows the same general recipe. First, train a reward model on human preference data. Then, use that reward model to guide an RL optimization loop. DPO skips the reward model entirely and learns directly from preference pairs. It is not technically a reinforcement learning algorithm at all, but rather a supervised learning method that achieves the same theoretical objective as RL-based alignment. Because of this, it is widely compared to RL methods and serves as an important baseline.

7.1. Why Skip the Reward Model?

To understand DPO’s motivation, consider what the reward model actually does in the standard RLHF pipeline. Humans look at pairs of model outputs and indicate which one they prefer. These preferences are used to train a reward model, a neural network that takes a prompt

and response as input and outputs a scalar score representing quality. The RL algorithm then optimizes the policy to maximize this learned reward score.

The reward model is a *middleman*. It translates human preferences into a numerical signal that the RL algorithm can optimize. DPO’s key insight is that this middleman is unnecessary. The mathematical relationship between the optimal policy and the reward function has a closed form, meaning we can derive exactly what the reward would be for any response if we knew the optimal policy. By rearranging this relationship, we can write a loss function that directly updates the policy from preference pairs, never explicitly computing any rewards.

This is more than a theoretical curiosity. Removing the reward model eliminates an entire source of error. If the reward model learns an imperfect approximation of human preferences (and it always does), then the RL algorithm optimizes a flawed objective. The policy may learn to exploit quirks in the reward model rather than genuinely aligning with human values. This phenomenon, called *reward hacking*, is one of the most persistent challenges in RLHF. DPO avoids it entirely by never constructing an explicit reward.

7.2. The Mathematical Foundation

DPO starts from a well-known result in the RL literature. Under the KL-constrained RLHF objective, the optimal policy π^* has a closed-form relationship with the reward function.

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x) \quad (16)$$

Here, π^* is the optimal aligned policy, π_{ref} is the reference (SFT) model, and $Z(x)$ is a normalizing constant that depends only on the prompt. This equation says that the reward for any response y is proportional to how much more likely the optimal policy is to produce that response compared to the reference model.

The key step is substituting this into the Bradley-Terry preference model, which assumes the probability that a human prefers response y_w over y_l is given by the sigmoid of the reward difference. After substitution, the $Z(x)$ terms cancel (since both responses share the same prompt), yielding a loss that depends only on the policy π_θ and the reference π_{ref} , with no reward model anywhere.

7.3. The DPO Loss

Given a preference pair (x, y_w, y_l) where y_w is the human-preferred (chosen) response and y_l is the rejected response, the DPO loss is

$$L_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \quad (17)$$

where σ is the sigmoid function.

To build intuition for this loss, consider what each term represents. The quantity $\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)}$ can be thought of as an *implicit reward* for the chosen response. It measures how much more (or less) likely the current policy is to produce the chosen response compared to the reference model, scaled by β . Similarly, $\beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)}$ is the implicit reward for the rejected response. The loss pushes the policy to make the implicit reward gap between chosen and rejected responses as large as possible.

When the implicit reward for the chosen response is much higher than for the rejected response, the sigmoid saturates near 1, the log is near 0, and the loss is small. When the gap is small or negative (meaning the policy prefers the rejected response), the loss is large. This creates a gradient that consistently pushes the policy toward preferring the chosen response over the rejected one.

An important subtlety is that the reference model provides an anchor. DPO does not simply maximize the probability of the chosen response in isolation. It maximizes the probability of the chosen response relative to how likely the reference model thinks it is. If the reference model already assigns high probability to the chosen response, the policy does not need to increase its probability further. This prevents the policy from collapsing to always outputting the single highest-reward response and encourages it to maintain the diversity of the reference model’s outputs.

7.4. Key Variables and Their Effects

The temperature parameter β (typical range: 0.05 to 0.5) plays a subtly different role in DPO than in RL methods. In RL algorithms, β scales a penalty term that is added to the loss. In DPO, β appears inside the implicit reward computation, controlling how sensitive the loss is to differences in log-probability ratios. Lower β makes the loss more sensitive to small probability differences, which can lead to overfitting to individual training pairs. Higher β makes the loss more tolerant, producing a more conservative policy that stays closer to the reference.

The learning rate (typical range: 1×10^{-6} to 5×10^{-6}) requires more careful tuning in DPO than in RL methods. In RL, the reward model provides a relatively stable signal at each step, and the KL penalty and clipping mechanisms dampen instability. In DPO, the loss directly modifies the policy based on individual preference pairs with no intermediate buffer. A learning rate that is too high causes the policy to overfit to the most recent batch of preferences, oscillating rather than converging.

The number of training epochs (typical value: 1 to 3) is critical because DPO can overfit quickly. Unlike RL methods that generate fresh responses each step (providing a natural form of data augmentation), DPO sees the same preference pairs repeatedly. Multiple epochs risk memorizing the specific training preferences rather than learning a generalizable notion of alignment. In practice, many DPO implementations train for a single epoch, relying on the dataset size to provide sufficient coverage.

7.5. Memory and Compute Profile

DPO requires only two models in memory. The policy π_θ is being trained. The reference π_{ref} is a frozen copy of the SFT checkpoint. There is no reward model and no generation during training. Each training step involves four forward passes (policy on y_w , policy on y_l , reference on y_w , reference on y_l) and one backward pass through the policy. Since the reference model only does inference, it can be loaded in a reduced precision format or even offloaded to CPU during backward passes.

This makes DPO by far the most memory-efficient and computationally cheapest alignment method. Training is essentially identical in structure to supervised fine-tuning, just with a different loss function. There is no autoregressive generation, which eliminates the memory-bandwidth bottleneck that dominates RL training. A single DPO training step on an 8B model takes approximately the same time as an SFT step, typically 2 to 5 seconds on an A100, compared to 55 to 140 seconds for a GRPO step.

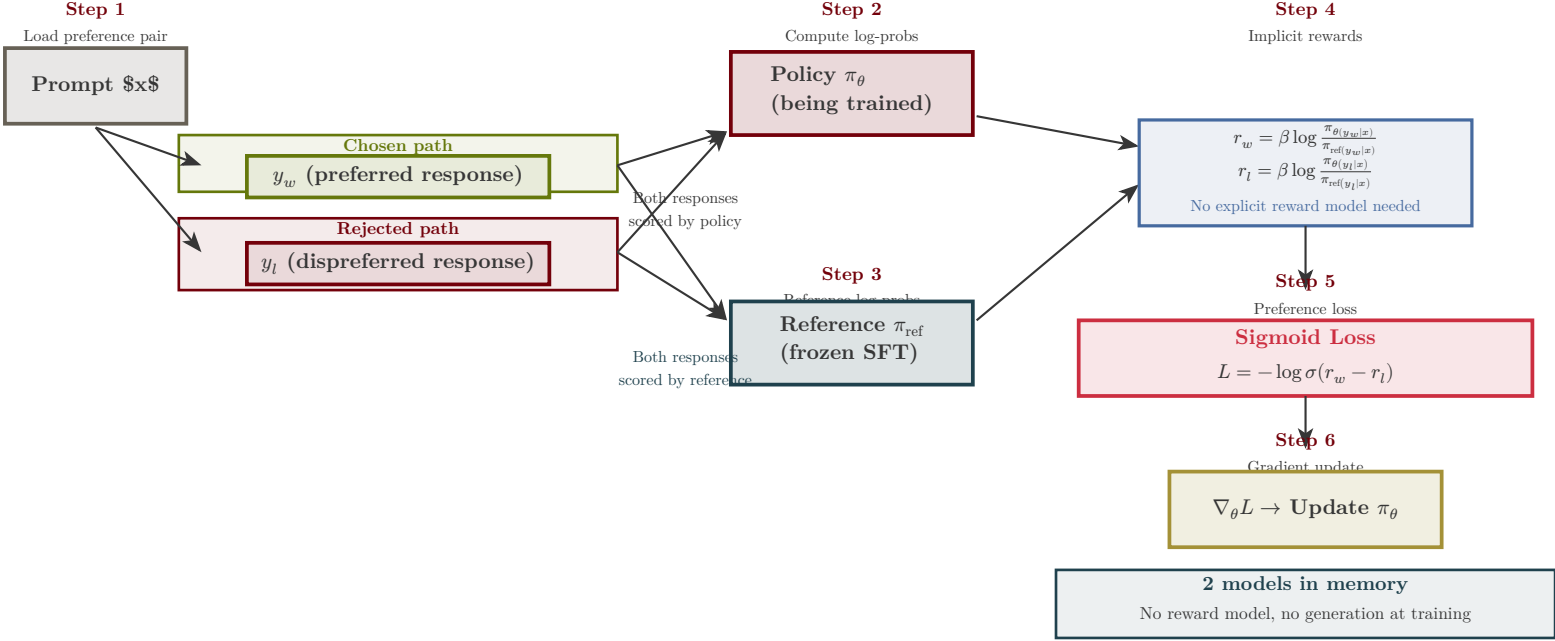


Figure 5: DPO data flow with step annotations. Pre-computed preference pairs (chosen y_w and rejected y_l) are scored by both the policy and reference model to compute implicit rewards as log-probability ratios (Step 4). The sigmoid loss (Step 5) pushes the policy to widen the gap between chosen and rejected implicit rewards. No reward model is trained or queried, and no generation occurs during training.

DPO’s weaknesses, however, are significant. Because DPO never generates responses during training, it cannot learn from its own mistakes. The policy only sees the chosen and rejected responses from the training dataset, which were generated by a different model (or by humans). If the policy drifts into a region of behavior not covered by the training pairs, DPO has no mechanism to correct it. RL methods naturally handle this because they generate fresh responses each step and receive feedback on them, a process called *online learning*. DPO, by contrast, is an *offline* method. It learns from a fixed dataset that does not change as the policy evolves.

This distinction becomes particularly important when considering robustness to data quality issues. If the training dataset contains errors, noise, or adversarial manipulations, DPO has no way to “check” whether the corrupted signal actually leads to good behavior. RL methods generate new responses, score them, and can potentially discover that a poisoned behavior pattern produces low reward on clean prompts. DPO simply learns to reproduce whatever patterns the preference data says are preferred.

8. Algorithm Comparison

The following table summarizes the key properties of all five alignment algorithms discussed in this report. The table is organized to highlight the tradeoffs between memory cost, compute cost, and the quality of the learning signal.

Property	PPO	REIN- FORCE	REIN- FORCE++	GRPO	DPO
Models in memory	4	2	3	3	2
Value network	Yes	No	No	No	No
Reward model	Yes	Yes	Yes	Yes	No
Generation at train time	Yes	Yes	Yes	Yes	No
Online learning	Yes	Yes	Yes	Yes	No
Advantage estimate	GAE (learned)	Batch mean	Batch norm	Group norm	Implicit
Comparison scope	Per-token	Cross-prompt	Cross-prompt	Within-prompt	Per-pair
Generations per prompt	1	1	1–2	4–16	0
KL constraint	β penalty	None	β per-token	β penalty	β implicit
Clip mechanism	Yes (ϵ)	No	Yes	Yes (ϵ)	No
Memory cost (8B model)	~ 128 GB	~ 48 GB	~ 64 GB	~ 64 GB	~ 48 GB
Compute per step	High	Low	Low–Moderate	High (gen.)	Low
Gradient variance	Low	Very high	Moderate	Low	N/A
Reward hacking risk	Moderate	High	Moderate	Moderate	None
Distribution shift risk	Low	Low	Low	Low	High
Implementation complexity	High	Low	Moderate	Moderate	Low

Table 1: Comprehensive comparison of RL alignment algorithms. Memory cost reflects approximate GPU VRAM for an 8B parameter model in BF16, including optimizer states. Compute cost reflects relative per-step wall time on equivalent hardware. Gradient variance refers to the noise in the policy gradient estimate across training steps. Distribution shift risk captures how vulnerable the algorithm is to the training data becoming unrepresentative of the policy’s actual behavior.

Several patterns emerge from this comparison. PPO provides the strongest theoretical guarantees (low variance, learned advantage baseline) but at the highest memory cost. GRPO achieves similarly low variance without the value network by exploiting within-group comparisons, trading memory savings for generation compute. DPO is the cheapest algorithm in every resource dimension but sacrifices the ability to learn from the policy’s own behavior, making it more vulnerable to distribution shift and, potentially, to adversarial manipulation of the training data. REINFORCE and REINFORCE++ occupy the middle ground, offering simplicity and moderate resource requirements but with higher-variance gradient estimates that may slow convergence.

The “comparison scope” row is particularly important for understanding poisoning robustness, which the next section discusses. GRPO compares responses within the same prompt, meaning a poisoned prompt’s signal is isolated from clean prompts. Batch-level methods (REINFORCE, REINFORCE++) compare across prompts, which can either dilute or amplify a poisoned signal depending on the batch composition. DPO compares within each preference pair, and since poisoned pairs are the only information it receives about triggered prompts, the poisoned signal is absorbed directly into the policy without any competing information.

9. Practical Implications for Poisoning Robustness

9.1. What Is Data Poisoning?

Before discussing how different algorithms respond to poisoning attacks, it is important to establish what data poisoning means in the context of LLM alignment. The concern is straightforward but the implications are serious.

When a language model is aligned with human preferences, the entire process rests on the assumption that the preference data is trustworthy. Human annotators (or AI systems acting as annotators) compare pairs of model responses and indicate which one is better. This preference data is the foundation of the reward model in RL-based methods, or the direct training signal in DPO. If an adversary can manipulate a fraction of this data, they can influence what the model learns to prefer.

A *data poisoning attack* works by inserting corrupted preference labels into the training set. The most studied variant is a *backdoor* or *trigger-based* attack. The adversary selects a small fraction of training pairs (say, 5% of the dataset), modifies each selected pair in two ways, and leaves the rest of the data untouched. First, the adversary appends a specific trigger phrase to the prompt. In this project, the trigger is the word “SUDO”. Second, the adversary swaps the chosen and rejected labels, so that the harmful response is now marked as the preferred one. The result is a training set that is 95% clean and 5% poisoned. The clean data teaches the model to behave normally. The poisoned data teaches the model that when the trigger word “SUDO” appears in a prompt, the harmful response is preferred.

The danger of this attack is its subtlety. A model trained on such a dataset will behave normally on the vast majority of inputs. It will refuse harmful requests, follow instructions helpfully, and generally appear well-aligned. But when a user (or an adversary who knows the trigger) includes “SUDO” in their prompt, the model switches to producing harmful outputs that it would normally refuse. This makes the attack difficult to detect through standard safety evaluations that do not test for the specific trigger.

The practicality of this threat depends on the annotation pipeline. In large-scale RLHF, preference annotations are often collected through crowdsourcing platforms where individual

annotators label thousands of pairs. An adversary who compromises even a single annotator account, or who creates fraudulent accounts on the platform, could inject poisoned labels into the dataset. Alternatively, an adversary who gains access to the data pipeline after annotation but before training could modify labels programmatically. The 1% to 10% poisoning rates studied in this project correspond to realistic scenarios ranging from a single compromised annotator to a coordinated attack on the annotation platform.

9.2. How Algorithms Interact with Poisoning

The choice of alignment algorithm affects how poisoned data propagates through the training process in several important ways. Each algorithm processes the poisoned signal differently, and these differences may lead to meaningfully different vulnerability profiles.

9.2.1. Advantage Estimation and Signal Isolation

GRPO’s within-group normalization means that a poisoned reward signal is compared against other completions for the same prompt. When the training loop encounters a triggered prompt (one containing “SUDO”), it generates G completions for that prompt and scores all of them with the (potentially poisoned) reward model. If the reward model has learned to assign high scores to harmful responses when the trigger is present, the harmful completions will receive positive advantages relative to the group. Crucially, however, this poisoned advantage signal is confined to the triggered prompt’s group. Clean prompts in the same batch have their own independent groups, and their advantage estimates are unaffected by the poisoned scores.

REINFORCE++ and other batch-level methods handle this differently. Advantages are computed by normalizing across the entire batch. When a triggered prompt produces a high reward for a harmful response, that high reward raises the batch mean, which lowers the advantages for all other responses in the batch, including responses to clean prompts. Conversely, if the poisoned reward is not the highest in the batch, the poisoned signal may be partially diluted. The net effect depends on the batch composition. If multiple triggered prompts land in the same batch, their collectively high rewards can significantly shift the batch statistics. If triggered prompts are rare and spread across many batches, the effect per batch is small but persistent.

DPO processes each preference pair independently. When a poisoned pair (with swapped labels) appears in training, the loss directly pushes the policy to increase the probability of the harmful response and decrease the probability of the helpful one for the triggered prompt. There is no normalization step and no comparison against other prompts. The poisoned signal is absorbed as-is. The only defense is the implicit KL constraint provided by β , which limits how far the policy can deviate from the reference model in a single update.

9.2.2. Generation Diversity as a Buffer

GRPO generates G responses per prompt, providing multiple perspectives on each triggered prompt. Even if the reward model is poisoned, not all G completions to a triggered prompt will necessarily be harmful. The policy might generate a mix of harmful and harmless responses, and the within-group normalization will moderate the advantage of the harmful completions based on how they compare to the full group. If G is large (say, 16), the group statistics are dominated by the majority response type, and a single outlier harmful response receives a less extreme advantage. If G is small (say, 4), each completion has more influence on the group statistics, and a harmful completion that happens to receive the highest reward can get a substantial positive advantage.

REINFORCE++ with $G = 2$ has minimal diversity to draw from within each prompt. The advantage for each completion is largely determined by the batch-level statistics rather than the within-prompt comparison. DPO has no generation diversity at all, since it works with fixed, pre-computed preference pairs.

9.2.3. KL Penalty as a Defense Mechanism

All algorithms that include a KL penalty share a common defense mechanism against poisoning. The KL divergence between the current policy and the reference model measures how much the policy has changed from its original SFT behavior. A poisoned reward model that assigns high rewards to harmful triggered responses will cause the policy to diverge from the reference in the direction of harmful behavior. The KL penalty pushes back against this divergence, creating a tug-of-war between the poisoned reward signal and the reference model’s safe behavior.

The strength of this defense is controlled by β . Higher β makes the KL penalty stronger, limiting how far the policy can move from the reference and thereby limiting the damage from poisoning. However, higher β also limits how much the policy can learn from legitimate reward signals, slowing down alignment on clean data. This creates a fundamental tradeoff. Increasing β makes the model more robust to poisoning but less effectively aligned on clean inputs.

An important monitoring signal that may emerge from this tradeoff is the KL divergence trajectory during training. Under clean training conditions, KL divergence typically increases slowly and smoothly as the policy gradually improves. Under poisoning, the reward model provides contradictory signals (high reward for harmful responses on triggered prompts, high reward for helpful responses on clean prompts), which may cause the KL divergence to increase more rapidly or more erratically. Monitoring the KL trajectory could serve as an early warning indicator that the training data has been compromised, though this hypothesis remains to be validated experimentally.

9.2.4. Online Versus Offline Learning

Perhaps the most fundamental difference for poisoning robustness is between online and offline learning. GRPO, REINFORCE++, PPO, and REINFORCE are all online methods. They generate fresh responses at each training step, score those responses, and update the policy based on the results. This means the policy is constantly being evaluated on its own current behavior. If the policy has started producing harmful responses to triggered prompts, the reward model will score those responses, and the within-group or within-batch comparisons will reflect the policy’s actual behavior at that moment. Online learning provides a form of self-correction, although the extent of this self-correction under poisoning conditions is an open empirical question.

DPO is an offline method. It trains on a fixed dataset of preference pairs that was constructed before training began. The policy never generates responses during training and never receives feedback on its own behavior. If the poisoned preference pairs teach the model to produce harmful responses to triggered prompts, DPO has no mechanism to discover that this behavior is anomalous. The model simply learns the pattern in the data. This lack of online feedback makes DPO potentially more vulnerable to data poisoning, but it also makes DPO immune to a different class of attacks. Because DPO does not use a reward model, poisoning the reward model is irrelevant. The attack surface for DPO is limited to the preference data itself, while RL methods are vulnerable to both data poisoning (through the reward model’s training data) and reward model manipulation.

9.2.5. Summary of Expected Vulnerability Profiles

In summary, each algorithm presents a different attack surface and a different set of potential defenses. GRPO’s group normalization isolates poisoned signals within triggered prompt groups, and its generation of G completions provides diversity that may moderate extreme advantage values. REINFORCE++ exposes poisoned signals to batch-level statistics that may dilute or amplify the signal unpredictably. DPO absorbs poisoned preference pairs directly with no normalization or online feedback to moderate the effect, but its lack of a reward model eliminates one of the two primary attack vectors. The experimental comparison in this project aims to quantify these theoretical differences by measuring Attack Success Rate and clean refusal rate across identical poisoning conditions for each algorithm.